

팀 프로젝트

Employee Management System | C++17 중심 · Java 참고

VSCode C++17 & Java 개발자 과정 | 5일 × 8교시 팀 프로젝트

SPEC 단계

요구사항·PRD
To-Do

100점

요구사항 평가

RED 단계

실패하는
테스트 먼저

400점

s/w 품질 평가

GREEN 단계

최소 구현으로
통과

100점

BP 발표

REFACTOR

품질·설계
개선

500점

총점

TRACEABILITY : Invariant → UI Contract Test → Domain Test → Implementation Unit

1팀 4~5인 | 팀 내 Code Review 필수 | TDD Practice 필수 | Commit 50줄 이하 권장

5일 일정 타임라인

전체 일정

1일차	2일차	3일차	4일차	5일차
<i>1~3교시</i> Team OT·팀 빌딩 그라운드 룰	<i>1~4교시</i> PHASE 1 — RED 실패하는 테스트 먼저 작성	<i>1~4교시</i> PHASE 2 — GREEN 최소 구현으로 테스트 통과	<i>1~4교시</i> PHASE 3 — REFACTOR Clean Code·설계 개선	<i>1~5교시</i> 팀 프로젝트 회고 발표 자료 정리
<i>3~5교시</i> C++ & JavaBase code 분석 요구사항 분배	<i>4~8교시</i> 요구사항별 TC 작성 Test 커밋	<i>4~8교시</i> 구현 완성·PR Review ctest 통과 확인	<i>4~8교시</i> 통합·BP 문서 작성 최종 빌드 검증	<i>6~8교시</i> 팀별 발표 (BP 포함)
<i>5~8교시</i> SPEC: PRD.md·README To-Do List 작성				
Invariant	→ UI Contract Test	→ Domain Test	→ Implementation Unit	

프로젝트 요구사항 & 채점 기준 (총 500점)

📋 프로젝트 기본 요구사항

📁 C++ & Java 사원 관리 DB 구현

직원번호·성명·직급·전화번호·생년월일 포함
CSV(txt) 파일 기반 영속성

👥 팀 구성

1팀 4~5인 | 팀 내 코드 리뷰 필수
GitHub Organization + Repository

🔄 TDD Practice 필수

RED→GREEN→REFACTOR 사이클 엄수
Google Test 기반 (참고: JUnit 5)

📄 커밋 50줄 이하 권장

작은 단위로 자주 커밋 | Conventional Commits
여러 커밋 → 하나의 PR 가능

🌿 Release 브랜치 제출

형식: A팀-27번-RED / GREEN / REFACTOR
단계별 PR & Review 필수

📊 채점 항목 (총 500점)

① 요구사항 평가

100점

• docs/PRD.md 파일 유/무

25점

• README.md 파일 유/무

25점

• To-Do List 정리 유/무

50점

② s/w 품질 평가

400점

• 테스트 코드 유/무

100점

• 단계별 브랜치 4개 × 50점

200점

• 단계별 PR 리뷰 10개↑ × 50점

200점

③ BP 평가

100점

• 프로젝트 내 BP 문서 유/무

50점

• BP 발표 유/무

50점

SPEC 단계 PRD · README To-Do List

요구사항 문서화 | 브랜치: 팀-번호-SPEC

⚙ C++17 (주)

☪ Java 17 (참고)



SPEC 단계 — 브랜치 전략 & 필수 문서 작성

Repository & 브랜치 전략 (C++ & Java 프로젝트 기준)

```
# C++ & Java 프로젝트 — Git 브랜치 설정
# Organization: TeamProject_A팀
# Description: #1팀, 홍길동, 박문수, 이순신, 황진희

git checkout -b A-27-SPEC      # SPEC 단계
git checkout -b A-27-RED      # RED 단계
git checkout -b A-27-GREEN    # GREEN 단계
git checkout -b A-27-REFACTOR # REFACTOR 단계

# main merge 시 PR & Review 필수
# PR 당 리뷰 10개 이상 달성 목표
```

필수 문서 3종 (채점 항목 ①)

docs/PRD.md (25점)

프로젝트 목적 & 배경 | C++ & Java 기능 요구사항 목록 | 비기능 요구사항 (성능·보안) | DB 스키마 정의 (struct Employee)

README.md (25점)

프로젝트 개요·팀원 소개 | C++ & Java 빌드 & 실행 방법 (CMake) | Java 병행 빌드 (Maven) 참고 | 디렉토리 구조 & To-Do List

To-Do List (50점)

요구사항별 체크박스 목록 | 우선순위 & 담당자 배정 | 완료 여부 실시간 갱신 | README.md 내 포함 필수

✓ To-Do List 형식 예시 (README.md — C++ & Java 기준, 채점 50점)

```
## 📄 To-Do List (README.md)
```

요구사항	담당자	단계	상태	브랜치
[ADD] 직원 추가	홍길동	GREEN	✓	A-27-GREEN
[DEL] 직원 삭제	박문수	RED	🔄	A-27-RED
[SCH] 이름 검색	이순신	SPEC	📄	-
[MOD] 정보 수정	황진희	SPEC	📄	-
[CNT] 수 조회	팀원	SPEC	📄	-

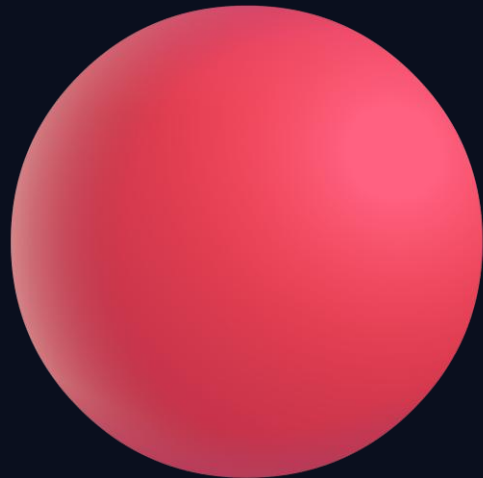
```
# C++ & Java 실제 TODO 항목 (README.md 내)
```

- [] certi column 추가 구현 & TC
- [] -p 출력 옵션: 최대 5개 / NONE
- [] MOD 기능 TC & 구현
- [] SCH 3옵션 부등호 (-g, -ge, -s, -se)
- [] AND/OR 다중 조건 (-a, -o)

🔴 SPEC 커밋: git commit -m "docs: PRD.md C++ & Java 요구사항 초안" | "docs: README.md To-Do List 추가"

연습 #팀 프로젝트

RED 단계 실패하는 테스트 먼저 작성



TDD 첫 번째 사이클 | 브랜치: 팀-번호-RED

⚙ C++17 (주)

☯ Java 17 (참고)

RED 단계 — 실패하는 테스트 먼저 작성 (C++ & Java 중심)

● 목표: 구현 없이 테스트 코드만 작성 → 컴파일/빌드 실패 또는 테스트 실패 확인 → 커밋

🔗 C++17 — Google Test RED 단계 (주)

```
// EmployeeManagerTest.cpp (RED - 구현 없이)
#include <gtest/gtest.h>
#include "EmployeeManager.h" // ← 미구현 → 컴파일 실패

class EmployeeManagerTest : public ::testing::Test {
protected:
    EmployeeManager manager;
    // ← 컴파일 실패
};

TEST_F(EmployeeManagerTest, ShouldFindEmployee_WhenAdded) {
    Employee emp{"15123456", "홍길동", "CL3", "010-1234-5678", "19900101"};
    manager.add(emp);
    auto result = manager.
findByNo("15123456");
    ASSERT_TRUE(result.has_value());

    EXPECT_EQ(result->name, "홍길동");
}

TEST_F(EmployeeManagerTest, ShouldNotFind_WhenDeleted) {
    manager.add({"15123456", "홍길동", "CL3", "010-1234-5678", "19900101"});
    int deleted = manager.
deleteByNo("15123456");
    EXPECT_EQ(deleted, 1);

    EXPECT_FALSE(manager.findByNo("15123456").has_value());
}

// cmake --build build → 컴파일 에러 → 커밋
```

🔗 Java — JUnit 5 RED 단계 (참고)

```
// EmployeeManagerTest.java (RED)

@DisplayName("직원 관리 - RED")
class EmployeeManagerTest {

    @BeforeEach void setUp() {
        manager = new EmployeeManager(); // 미구현
    }

    @Test
    @DisplayName
    ("ADD 후 조회")
    void
should_Find_WhenAdded() {
        Employee emp = new Employee(
            "15123456", "홍길동",
            "CL3", "010-1234-5678", "19900101");
        manager.add(emp);

        assertThat(manager
            .findByNo(
                "15123456"))
            .isPresent();
    }

    // mvn test → FAIL → 커밋
```

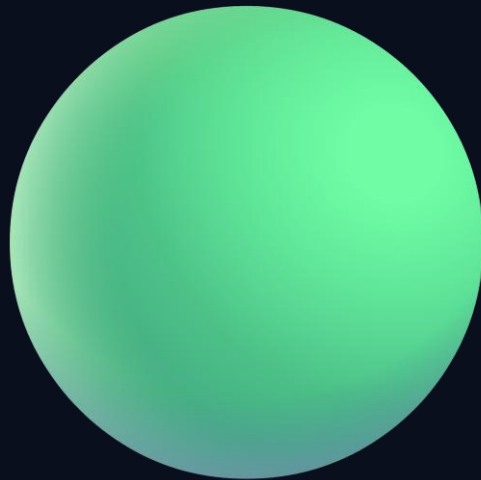
● RED 커밋: git commit -m "test: [ADD] 직원 추가 TC — RED" | 테스트가 실패하는 상태 그대로 커밋하세요. 이것이 TDD의 시작입니다.

GREEN 단계 최소 구현으로 테스트 통과

TDD 두 번째 사이클 | 브랜치: 팀-번호-GREEN

⚙ C++17 (주)

☪ Java 17 (참고)



GREEN 단계 — 최소 구현으로 테스트 통과 (C++ & Java 중심)

● 목표: 테스트가 통과하는 최소한의 코드 작성 — 품질보다 통과 우선, 중복·하드코딩 허용

⚙ C++17 — 최소 구현 GREEN (주)

```
// Employee.h + EmployeeManager.h (최소 구현)

// Employee.h
struct
Employee {
    std::string employeeNo, name;
    std::string position, phone, birthday;
    std::string certi; // ADD: certi 필드 (README TODO 반영)
};
```

🕒 Java — 최소 구현 GREEN (참고)

```
// Employee.java (Java Record)

public record Employee(
    String employeeNo,
    String name,
    String position,
    String phone,
    String birthday,
    String certi) {}
```

● GREEN 커밋: `git commit -m "feat: [ADD] 직원 추가 최소 구현 — GREEN"` | 테스트가 모두 통과하는 상태에서 커밋하세요.

GREEN 단계 — 최소 구현으로 테스트 통과 (C++ & Java 중심)

● 목표: 테스트가 통과하는 최소한의 코드 작성 — 품질보다 통과 우선, 중복·하드코딩 허용

⚙ C++17 — 최소 구현 GREEN (주)

```
// EmployeeManager.h
class
EmployeeManager {
    std::vector<Employee> db_;
public:
    void
    add(const Employee& emp) {
        db_.push_back(emp);
    }
    std::optional<Employee>

    findByNo(const std::string& no) const {
        for (auto& e : db_)
            if (e.employeeNo == no) return e;
        return std::nullopt;
    }
    int
    deleteByNo(const std::string& no) {
        int prev = db_.size();
        db_.erase(std::remove_if(db_.begin(), db_.end(),
            [&](auto& e){ return e.employeeNo == no; }),
            db_.end());
        return prev - db_.size();
    }
};
// ctest → GREEN ✅ → 커밋
```

🔴 Java — 최소 구현 GREEN (참고)

```
// EmployeeManager.java
public class
EmployeeManager {
    private final List<Employee>
        db = new ArrayList<>();

    public void
    add(Employee emp) {
        db.add(emp);
    }
    public Optional<Employee>

    findByNo(String no) {
        return db.stream()
            .filter(e -> e.employeeNo()
                .equals(no))
            .findFirst();
    }
}
// mvn test → GREEN ✅
```

연습 #팀 프로젝트

REFACTOR 단계

Clean Code & 설계 개선

TDD 세 번째 사이클 | 브랜치: 팀-번호-REFACTOR

⚙ C++17 (주)

☪ Java 17 (참고)



REFACTOR 단계 — Clean Code & 설계 개선 (C++ & Java 중심)

● 목표: 테스트 Green 유지하면서 코드 품질 개선 — 중복 제거·네이밍·SRP·OCP 적용

⚙ C++17 — 리팩토링 포인트 (주)

```
// REFACTOR: 인터페이스 분리 + Modern C++17

// IEmployeeRepository.h (인터페이스 분리 - SRP)
class
IEmployeeRepository {
public:
    virtual ~IEmployeeRepository() = default;
    virtual void
save(const Employee&) = 0;
    virtual std::optional<Employee>

findByNo(const std::string&) const = 0;
    virtual std::vector<Employee>

findAll() const = 0;
    virtual int

deleteByNo(const std::string&) = 0;
    virtual std::vector<Employee>

findByName(const std::string&) const = 0;
};
```

🔗 Java — 리팩토링 포인트 (참고)

```
// REFACTOR: Repository 패턴

// EmployeeRepository.java (SRP)
public interface
EmployeeRepository {
    void
save(Employee emp);
    Optional<Employee>
findByNo(String no);
    List<Employee>
findAll();
    int
deleteByNo(String no);
    List<Employee>
findByName(String name);
}
```

REFACTOR 단계 — Clean Code & 설계 개선 (C++ & Java 중심)

● 목표: 테스트 Green 유지하면서 코드 품질 개선 — 중복 제거·네이밍·SRP·OCP 적용

⚙ C++17 — 리팩토링 포인트 (주)

```
// InMemoryEmployeeRepository.h
class
InMemoryEmployeeRepository

: public IEmployeeRepository {
    std::unordered_map<std::string, Employee> store_;
public:
    void
    save(const Employee& e) override {
        store_[e.employeeNo] = e;
    }
    // ... 나머지 override 구현
};
// ctest → GREEN 유지 → 커밋
```

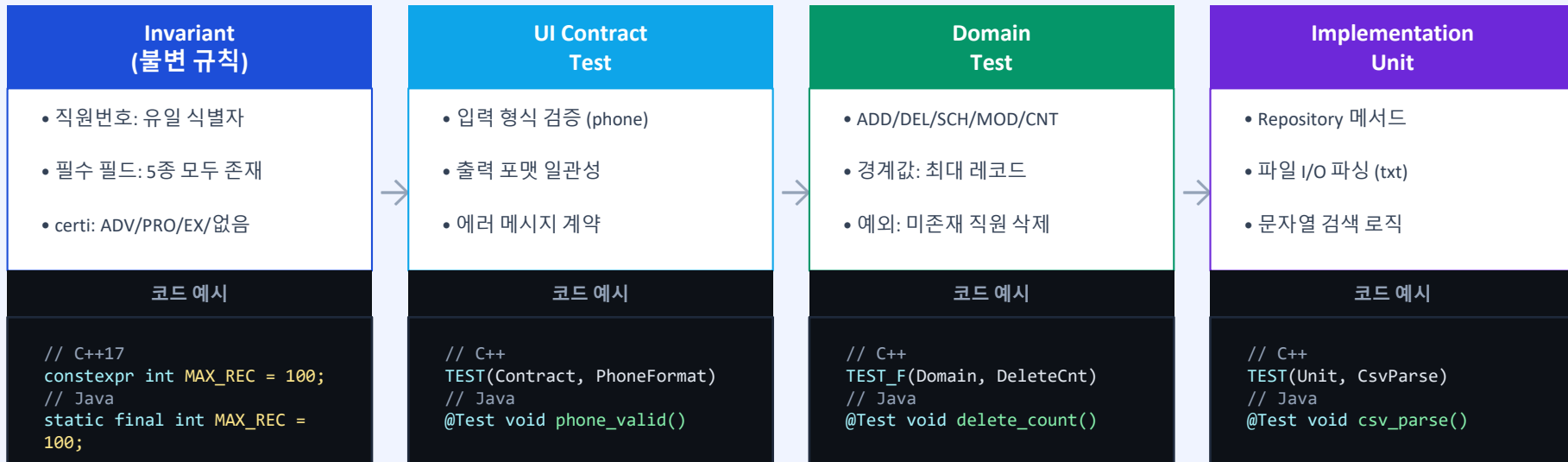
🕒 Java — 리팩토링 포인트 (참고)

```
// InMemoryEmployeeRepository.java
public class
InMemoryEmployeeRepository
    implements
    EmployeeRepository {
    private final Map<String, Employee>
        store = new LinkedHashMap<>();
    @Override public void
    save(Employee e){
        store.put(e.employeeNo(), e);
    }
    // ... 나머지 구현
}
// mvn test → GREEN 유지
```

● REFACTOR 커밋: `git commit -m "refactor: Repository 패턴 적용으로 저장소 분리"` | 리팩토링 후 반드시 `ctest / mvn test` Green 확인하세요.

TRACEABILITY — 요구사항 추적성 매핑

Invariant → UI Contract Test → Domain Test → Implementation Unit



요구사항 추적 매트릭스 (To-Do List 기반)

요구사항	Invariant	Contract	Domain Test	Unit
[ADD] 직원 추가	유일성 검증	입력 형식	add() TC	save() 메서드
[DEL] 직원 삭제	PK 존재 확인	에러 메시지	deleteByNo() TC	erase() 로직
[SCH] 검색 + 3옵션	필드 범위	출력 포맷	findByName() TC	파싱 로직
[MOD] 수정	불변 PK	변경 계약	modify() TC	update() 메서드

PR 리뷰 가이드 & 커밋 전략 (채점 S/W 품질 400점)

 PR 리뷰 10개 이상 달성 전략 (단계별 × 50점 = 200점)

✅ 코드 정확성

요구사항 충족·로직 오류·경계값 처리 확인

🔒 캡슐화·접근제어

C++: const 멤버 함수·private 필드 | Java: private/public 적절성

📁 네이밍 컨벤션

C++: snake_case 일관성 | Java: camelCase 통일

🧪 테스트 품질

C++ & Java TEST_F 명확성·경계값 포함 | Java @DisplayName

♻️ 중복 코드 DRY

추출 가능한 공통 메서드 제안

🔥 성능·메모리

C++: RAII 준수·불필요한 복사 | Java: 불필요 객체 생성

📖 Doxygen / Javadoc

public API 문서화 여부 확인

💡 리뷰 예시: "findByName 이 O(n) 인데 unordered_map 인덱싱 고려 어떨까요?"

 커밋 전략 (50줄 이하 권장)

Conventional Commits 형식

SPEC 단계

git commit -m

"docs: PRD.md C++ & Java요구사항 초안"

git commit -m "docs: README.md To-Do List 추가"

RED 단계

git commit -m

"test: [ADD] 직원추가 TC - RED"

git commit -m "test: [DEL] 직원삭제 TC - RED"

git commit -m "test: [SCH] 이름검색 TC - RED"

GREEN 단계

git commit -m

"feat: [ADD] 직원추가 구현"

git commit -m "feat: [DEL] 직원삭제 구현"

REFACTOR 단계

git commit -m

"refactor: IRepository 분리"

git commit -m "style: snake_case 네이밍 통일"

50줄 분할 예시:

ADD 기능 → 5개 커밋으로 분할

1)헤더 2)구현 3)TC 4)CMake 5)문서

🌿 브랜치 규칙 (채점 200점): A팀 27번 → A-27-SPEC | A-27-RED | A-27-GREEN | A-27-REFACTOR

VSCode 개발 환경 설정 & 핵심 요구사항 구현 예시

 .vscode/settings.json (공통)

```
{
  // C++ & Java 설정 (주)
  "C_Cpp.default.cppStandard": "c++17",
  "cmake.buildDirectory":
    "${workspaceFolder}/build",
  "clangd.arguments": ["--header-
insertion=never"],

  // Java 설정 (참고)
  "java.test.defaultConfig": "junit5",
  "editor.formatOnSave": true,
  "editor.tabSize": 4
}
```

 C++ & JavaCMake 빌드 (주)

```
# 빌드 & 테스트
cmake -B build
cmake --build build
cd build && ctest --output-on-failure

# 커버리지
lcov --capture --directory .
genhtml coverage.info -o coverage/

# 실행
./EmployeeManagement input.txt output.txt
```

 Java Maven (참고)

```
# 빌드 & 테스트
mvn compile
mvn test
mvn jacoco:report

# 커버리지 보기
# target/site/jacoco/index.html

# C++ & Java ↔ Java 비교
# cmake+ctest ↔ maven+junit
# lcov ↔ jacoco
```


VSCode 개발 환경 설정 & 핵심 요구사항 구현 예시

📄 핵심 요구사항 구현 — C++17 인터페이스 (IEmployeeRepository)

```
// C++17 순수 가상 함수 인터페이스
class
IEmployeeRepository {
public:
    virtual void
    save(const Employee&) = 0;                // [ADD]
    virtual int
    deleteByNo(const std::string& employeeNo) = 0;    // [DEL]
    virtual std::vector<Employee>
    findByName(const std::string& name) const = 0;    // [SCH]
    virtual std::vector<Employee>
    findByBirthDay(const std::string& bd) const = 0; // [SCH]
    virtual int
    modifyPhone(const std::string& no, const std::string& phone) = 0; // [MOD]
    virtual int
    modifyPosition(const std::string& no, const std::string& pos) = 0;    // [MOD]
    virtual std::vector<Employee>
    findAll() const = 0;                    // [CNT/PRINT]
};
// 각 메서드 → RED TC 먼저 → GREEN 구현 → REFACTOR (Java: EmployeeRepository interface 동일 구조)
```

BP 문서 작성 & 팀 발표 가이드 (채점 100점)

📄 BP(Best Practice) 문서 (50점) — bp/BP.md

1장 TDD 적용 경험

RED→GREEN→REFACTOR 사이클
배운 점
Google Test 픽처설계 경험
(Java: JUnit5 비교)

2장 Clean Code 사례

Before/After 코드 비교 + 개선 근거
C++: RAII-const correctness / Java:
Record-Optional

3장 팀 협업 경험

PR 리뷰 효과적 사례·그라운드 룰
성과
Offline 리뷰 → GitHub PR 온라인
기록

4장 AI 활용

프롭트 엔지니어링 우수 사례
Gemini CLI / Cursor AI (C++/Java 공
통)

5장 트러블슈팅

가장 어려웠던 버그 & 해결 과정
C++ & Java 컴파일 오류 / CMake
빌드 이슈

6장 현업 적용 계획

팀원별 3개월 액션 아이템
Clean Code → 실무 코드베이스
적용

📌 발표 구성 (50점) — 팀별 발표

① C++ & Java 프로
젝트 데모 (2분)

→ ② TDD 과정 시
연 (2분)

→ ③ 우수 프롭트
트 공유

→ ④ 팀 회고

📄 Gemini CLI / Cursor AI — BP 문서 자동 생성 프롭트

```
# Cursor Chat (Ctrl+L) 또는 Gemini CLI
@Codebase @bp/BP.md
[P] 시니어 C++17 개발자 관점의 기술 문서 작성자
[C] C++17 + Java 17 팀 프로젝트
5일 TDD + Clean Code + Refactoring 실습
[T] 팀 BP 문서를 6장 구조로 작성:
- C++ & Java Before/After 코드 비교 포함
- Google Test + JUnit5 비교 포함
- Gemini CLI 활용 프롭트 우수 사례
[F] Markdown + 코드 블록 | bp/BP.md 저장
```

📅 팀 그라운드 룰 (1일차)

• 매일 스탠드업 미팅 (15분): 어제/오늘/블로커

• PR 리뷰 24시간 이내 응답 | 코멘트 최소 2개

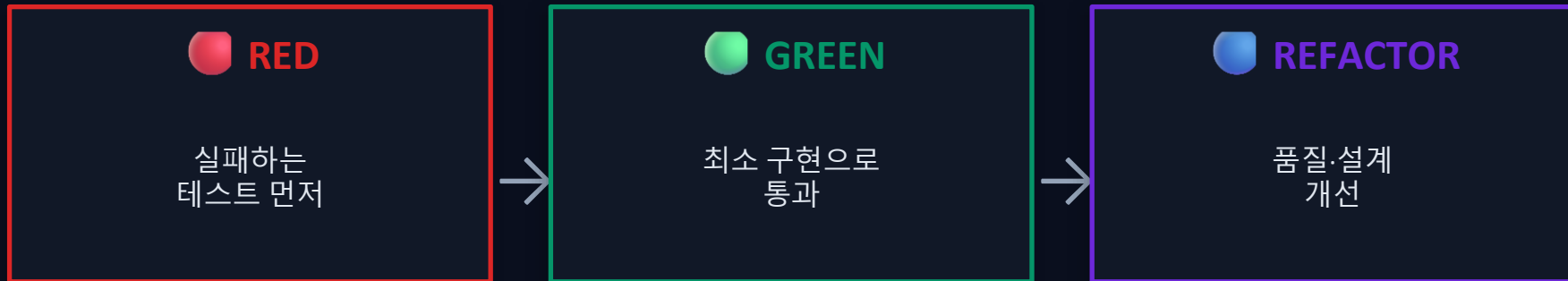
• 커밋 전 ctest / mvn test 실행 의무

• Cursor AI / Gemini CLI 제안 코드 함께 공유

• Offline 리뷰 → GitHub PR 코멘트 온라인 기록

📌 최종 제출 체크리스트

- ✓ Release 브랜치 생성 (A팀-27번-RED 형식)
- ✓ bp/BP.md 커밋 완료
- ✓ README.md To-Do List 갱신
- ✓ docs/PRD.md 최신화
- ✓ ctest 전체 통과 확인 후 제출



SPEC → RED → GREEN → REFACTOR: 각 단계 브랜치 & PR 리뷰 10개 이상이 핵심 점수 (400점)



PRD.md + README.md + To-Do List 3종 문서를 먼저 완성하면 방향이 잡힌다 (100점)



커밋 50줄 이하 + Conventional Commits: 작은 단위로 자주 커밋해야 히스토리가 살아있다



C++ & JavaTEST_F / Java @ParameterizedTest — 언어가 달라도 TDD 사이클은 동일하다



BP 문서와 발표로 팀의 성장 스토리를 기록하라 (100점) — 과정이 곧 결과물이다